

PRACTICAL

1. Virtual Address

```
#include <stdio.h>
int main()
{
    int x=10;
    printf("Value of x=%d\n",x);
    printf("Virtual address of x=%p\n",(void *)&x);
    return 0;
}
```

OUTPUT:

```
shashank@LAPTOP-24KPR0R8:~$ gcc pgm.c -o pgm
shashank@LAPTOP-24KPR0R8:~$ ./pgm
Value of x=10
Virtual address of x=0x7ffebf2aa6a4
```

2. Address Space

```
#include <stdio.h>
#include <stdlib.h>
int global=10;
int bss_variable;
int main()
{
    int local=20;
    int *heap=malloc(sizeof(int));
    *heap=30;
    printf("Code address      :%p\n",(void *)main);
    printf("Data address       :%p\n",(void *)&global);
    printf("BSS address        :%p\n",(void *)&bss_variable);
    printf("Heap address       :%p\n",(void *)heap);
    printf("Stack address      :%p\n",&local);
    free(heap);
    return 0;
}
```

Output;

```
shashank@LAPTOP-24KPR0R8:~$ ./Addresspace
Code address      :0x5c50f1d6a1a9
Data address      :0x5c50f1d6d010
BSS address       :0x5c50f1d6d018
Heap address      :0x5c5126bc6010
Stack address     :0x7ffc8a1b55ac
```

3. Paging

```
#include <stdio.h>
int main()
{
    int page_table[4]={1,-1,3,-1};
    int page;
    printf("Enter page number (0-3): ");
    scanf("%d", &page);
    if(page_table[page]==-1){
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else{
        printf("Page is already in memory\n");
        printf("Physical frame = %d\n", page_table[page]);
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPR0R8:~$ gcc Page.c -o Page
shashank@LAPTOP-24KPR0R8:~$ ./Page
Enter page number (0-3): 2
Page is already in memory
Physical frame = 3
```

4. Demand paging

```
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number       = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~$ gcc pageadd.c -o pageadd
shashank@LAPTOP-24KPROR8:~$ ./pageadd
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150
```

5. MallocFree

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int *p;
    p=malloc(5*sizeof(int));
    if(p==NULL){
        printf("Memory allocation failed\n");
        return 1;
    }
    for(int i=0;i<5;i++){
        p[i]=(i+1)*10;
    }
    for(int i=0;i<5;i++){
        printf("%d",p[i]);
    }
    printf("\n");
    free(p);
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~$ gcc Mallocfree.c -o Mallocfree
shashank@LAPTOP-24KPROR8:~$ ./Mallocfree
1020304050
```

6. Memory Accessing

```
#include <stdio.h>
int main(){
    int *p=NULL;
    printf("Before accessing memory\n");
    *p=100;
    printf("After accessing memory\n");
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~$ gcc Memoryacc.c -o Memoryacc
shashank@LAPTOP-24KPROR8:~$ ./Memoryacc
Before accessing memory
Segmentation fault ./Memoryacc
```

